

Image Warping and Homographies

[Home](#)

Fixed Center of Projection and a Rotation

In this project, I'm going to be playing around with homographies, the process of warping an image with matrix multiplication to make an image look like it was shot from a different



perspective.

To start with, here
are the images that
we are going to
manipulate soon.



Another Pair of Images



Calculating Homographies

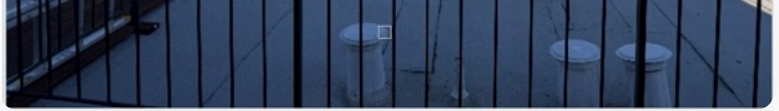
Homographies will be used here to warp one image into the perspective of the other image. The formula looks like this, $p' = Hp$.

A critical part of this is mapping points on one image to points on another. This is how we will actually retrieve the homography matrix.

We get these correspondences by using pointing and clicking on points on the two images that are the same.



Here is the calculated homography for this image. I will discuss how I got this matrix below.



Homography: $\begin{bmatrix} 2.59, & 0.0517, & -817.9, \\ 0.538, & 2.1, & -273, \\ 2.2 \cdot 10^{-3}, & 2.82 \cdot 10^{-5}, & 1 \end{bmatrix}$

Calculating Homographies

The homography matrix going from the bottom image to the top image looks like the following:

$$\begin{bmatrix} 1.866 & 2.52 \times 10^{-2} & -6.48 \times 10^{-2} & 0.293 & 1.55 & -174 \\ 8.7 \times 10^{-4} & -1.577 \times 10^{-5} & 1 \end{bmatrix}$$


System of

```
[p[i, 0], p[i, 1], 1, 0, 0, 0, -p_prime[i, 0] * p[i, 0], -p_prime[i, 0] * p[i, 1]]
[0, 0, 0, p[i, 0], p[i, 1], 1, -p_prime[i, 1] * p[i, 0], -p_prime[i, 1] * p[i, 1]]
```

System of Equations for Determining the Homographies

To determine the homography matrix, we first need to set up a linear system of equations mapping our corresponding points together.

We will create a matrix of rows as per this formula.

We create two rows for every single corresponding point.

The first is $[x, y, 1, 0, 0,$

$0, -x'y, -x'y]$, the second is $[0, 0, 0, x, y, 1, -y'x, -y'y]$. We stack all these rows on top of each other creating a matrix we will call A .

The minimum corresponding points we would need is 4, but since it would be nearly impossible to get the corresponding point exactly correct. I took nearly 20 corresponding points for my rectifications and panoramas.

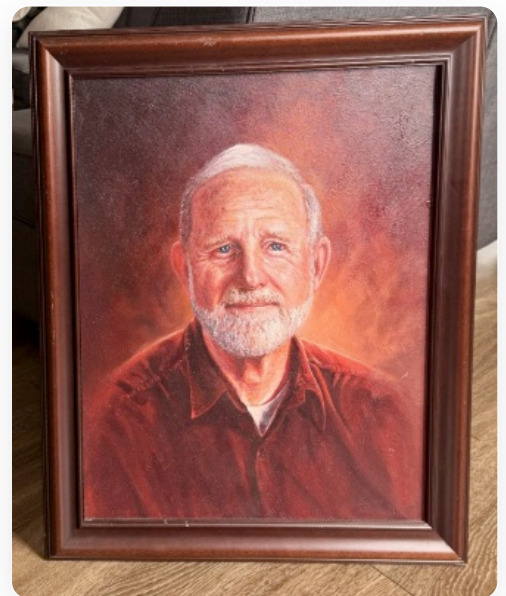
I then solved the system of equations using least squares which gives me a best fit solution. This is significantly more reliable than trying to perfectly select four

points.

Now for solving for H , we go ahead and set up $Ah = b$, where b is all of the corresponding points in the p' image we are calculating. The image with the new perspective

Warping Images

Now that we have homographies for our images, we can use them for warping.



First, we will try
this out on this
painting of
Rupert to see if
our
homography is
working.

First, we will
find some
points of
correspondence
on the original
image, then we
create some
values to map
those values to.

We want to
make these new
values be in the
shape of a
rectangle for
this part.

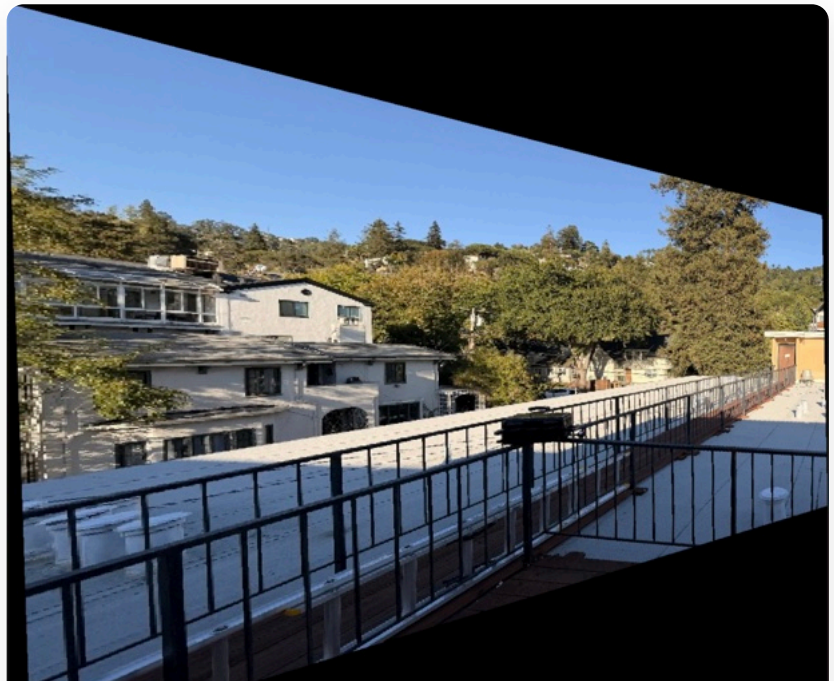
You can see

how initially we
were standing
to the side of
the painting by
a couple feet.

After
rectification, we
now see Rupert
as if we took
the picture
right in front of
him!

**Nearest
Neighbor
Interpolation
V Bilinear
Interpolation**

One result of of using



a homography is that we won't have a 1-1 pixel mapping and we will have to make a decision on what pixel values to utilize when constructing our new image.



There are a couple different ways we can interpolate pixel values when we are doing the inverse mapping of pixels.

One option is nearest neighbor which you can see on top here, while bilinear interpolation is on the bottom.

Nearest neighbor simply gets the

coordinate, e.g. (7.3, 2.5) and rounds to the nearest pixel. So you'll end up taking the value of (7, 3) from the original image for the pixel you're calculating in the new image. This is fine, but it leads to some jaggedness in the image. If you look at the tree on the left, you'll see what happens when there are similar colors around each other.

The railing also has picked up a lot of jagged edges.

Bilinear interpolation tries to solve this issue by calculating distance between the



location and neighbor pixels then it uses this distance to determine how much each neighboring pixel should contribute its pixel value to the result. Pixels that are farther will have less of an impact.

Bilinear interpolation takes the value of the pixel it lands in, the pixel to the right, pixel below, and pixel below and to the right in its calculations.

Everything looks significantly less jagged in the bilinear mapping, the railing looks straight, the tree is not *great* but



...is not great but

I can't make out
individual pixels like
in nearest neighbor.

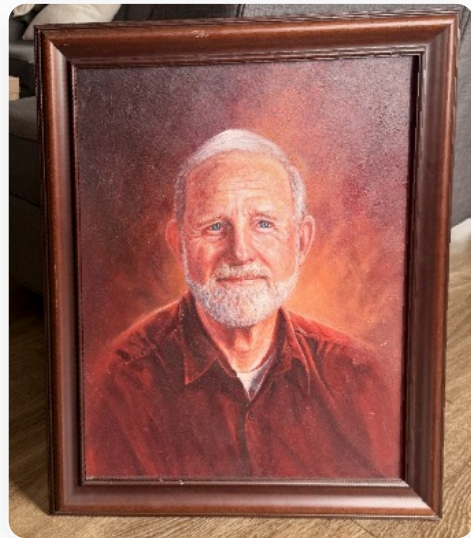
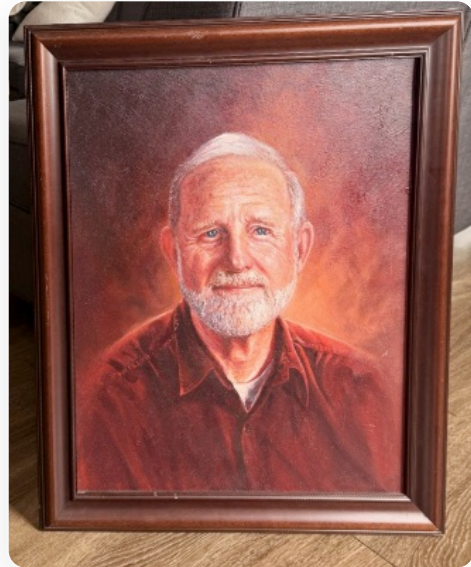
The cost of the
improved image is a
longer runtime. I am
using small images
that are about a
quarter the size of a
4k image, so it's not
terribly noticeable
here. But bilinear
interpolation has
several more
calculations
including
determining the pixel
values of the 3
additional neighbors
and then you need to
determine how much
weight the pixels
should have. So with
large images,
difference in runtime
will likely be pretty

noticeable.

Nearest Neighbor Interpolation V Bilinear Interpolation

We can see similar effects with the warping of Rupert with bilinear on top and NN on bottom.

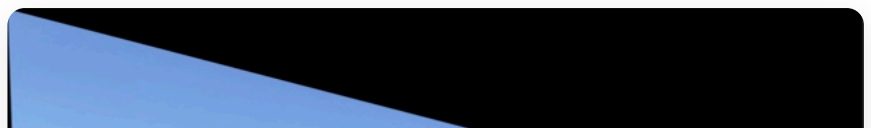
Specifically looking at the edges of the frame we can see nearest neighbor again making it a bit



jagged.

It's honestly pretty close in this image, likely because it's smaller. So, nearest neighbor definitely still has merits considering how much quicker it can be and how it can often make fairly okay results. That said, even here, benefits of bilinear interpolation and its longer runtime can be seen especially in the bottom right of the image.

Blending



Warped Images Into a Mosaic

In the previous project, we worked on splining images together to create seamless transitions between images. Here we will be doing a similar thing to blend our warped image into another regular image to create a sort of naive panorama.

I created a rectangular mask that spans the overlapped region between the two



images. The mask had a gradient applied to it so it picked up all of the left image on the left side and easefully transitioned to picking up all of the right image on the right side.

There is a slight bit of darkening at the top and bottom of the image where the mask was applied but there was no overlapping of the images. This can be rectified by adjusting the mask to more cleanly align with the overlapping region.

The source images for this transformation are presented above with their correspondences.

Utilizing Laplacian Stacks

To take advantage of the mask we just discussed, we use Laplacian stacks. Laplacian stacks allow us to have a list of photos, each with a certain band of frequencies. For our purposes here, since the images are quite similar



and we are carefully overlaying and aligning them. I only needed to use a laplacian stack of height 2.

We use the same formula as when we splined images together, like the Oraple.

$$\begin{aligned} \text{Hybrid_Level} = & \\ & \text{Gaussian_mask_level} * \\ & \text{Laplacian_image1_level} \\ & + (1 - \text{G_mask_level}) * \\ & \text{Laplacian_image2_Level} \end{aligned}$$

At the very end we will add up our hybrid levels and we have the splined together image. In our case, we result in a fairly viewable panorama.

One Last Image Mosaic

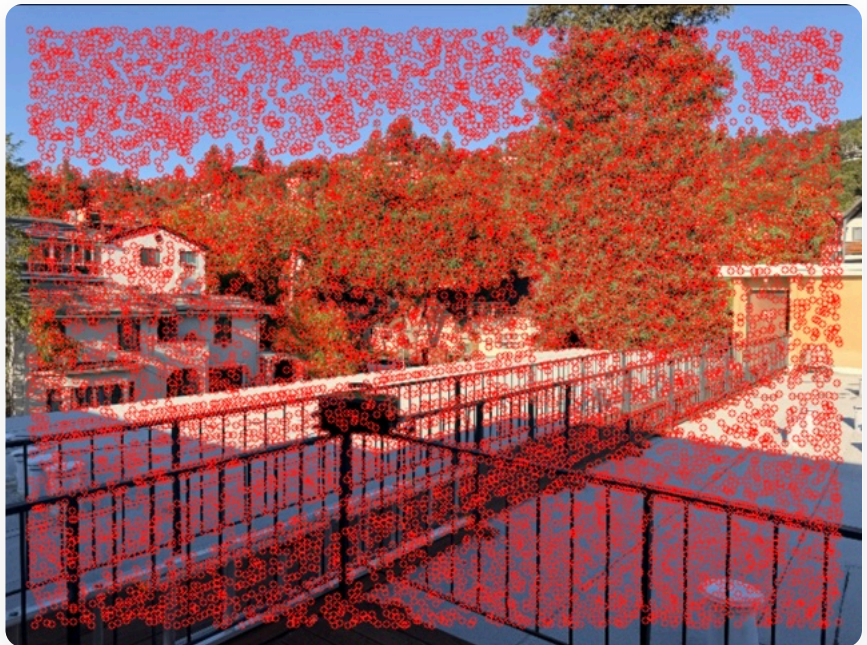


Here are
the source
images.

Resultant Panorama



Automatic Panorama Creation Using Harris Corner



Detection

Now that we see how we can manually create panoramas by finding correspondences and warping images, let's transition to finding an algorithm to automatically find correspondences.

The principal algorithm for this is Harris corner detection. Harris corner detection finds corners that will then be used as correspondences between our two images.



One problem with Harris corner detection is that it can give us a huge number of corners if we have busy images, specifically images with trees or brush. This often necessitates a suppression algorithm to reduce the number of detected corners.

Here, I'll compare regular Harris corner detection to Harris + Adaptive Non-Maximal Suppression which works to guarantee that interest points are spatially well distributed over an

image. The suppression algorithm will take the points in the Harris image that are local maxes by finding the points with the largest distance to points that are brighter than themselves.

Feature Descriptor Extraction

Once we have our most prominent corners, we want to get a patch around the



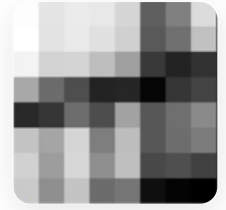
location of the corner so that we can better match corners to our other image. If we just use the value of the corner, it would be pretty likely that our other image would have several corners with a similar value.

Extracting patches of the corners however will capture some surrounding information around the corner and we will perform our corner matching based on these

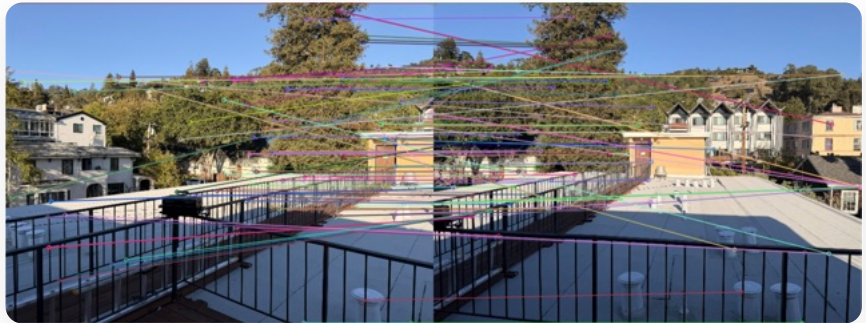
feature patches.

Here I started
with a 40 pixel
window around
the coordinate of
our corner. This
was then blurred
and
downsampled to
an 8x8 window.
This 8x8 window
is how we are
going to match
our corners.

A Few More Extractions



Matching Features



Using our Harris Detector + feature extractor, we will run both algorithms on two images that have similar points in frame. Now we will compare all of our extracted features from one

image to all the extracted features in our other image.

In order to determine if we have a match, we will just use Euclidean distance between the features. For each feature, we want to find two features from the other image that it pairs the best with.

If the 1st match is way better than the 2nd match, then this is a valid match and we keep it. If our 1st match is only slightly better, than it's likely neither are an



actually good match
and we discard
both.

Putting It All Together With RANSAC



While the feature
matching above
gets us pretty close,
there are some
matches which are
clearly not valid.

In order to correct
for these, we run

the RANSAC loop.

The RANSAC loop involves randomly sampling 4 matching points from the two images, computing the homography between them, and then evaluating whether that homography is actually any good.

We evaluate the performance of the homography by seeing how close a feature gets to its other matched feature. If it's within a reasonable distance, then we consider that match a true



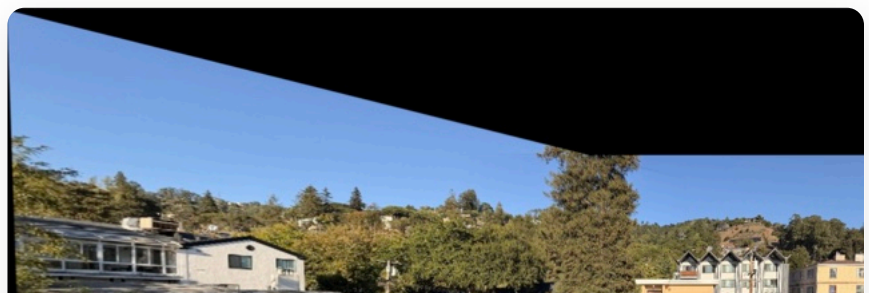
correspondence.

We run through this loop 5000 times in my implementation. At the end we will arrive with a list of points that are true matches, and we compute a final homography from these matches.

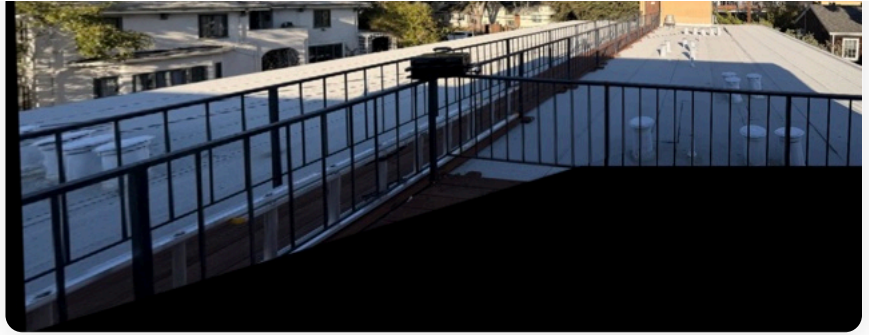
Finally, I use this automatically generated homography and use the same warping and stitching code from the first part of this project. Have a look at the results.

Comparing Manual panorama to Auto Panorama

Here we look at the first case, it's not super obvious if auto panorama is better here. I think where you can see some better performance is in the tree in the middle of the photo. The manual panorama just has a slight out-of-focus vibe to it.



But, is a great result nevertheless. I had to click on 20 points to do that manual panorama which took me probably like 2-3 minutes to find reasonable enough points that worked. With the auto panorama, I just ran main.py.



Comparing Manual panorama to Auto Panorama

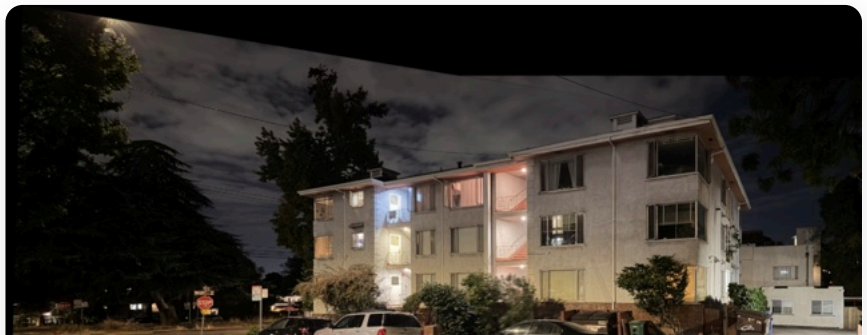


Again, this one was pretty good already in the original manual correspondence version.

But, if you look at the tree in the background, (the one next to the Campanile) the manual version has some significant fuzz that the autopano is able to successfully avoid.



Comparing Manual Panorama



to Auto Panorama

This version does have some good improvements that you can see without really putting your eye to it.

In the manual version, the asphalt looks quite blurry and almost like there is some motion blur going on. In the auto version, there is still a little bit of blur, but it's certainly improved.



<https://khody12.github.io/project3/index.html>